

POS System

Technical Documentation

Architecture & Implementation Guide
Version 1.0 | June 2026

Prepared by: POS System Engineering Team
Prepared for: Development, DevOps & Technical Teams

CONFIDENTIAL

This document contains proprietary technical information. Do not distribute without authorization.

Table of Contents

- Executive Summary
- System Architecture
 - 1. Project Overview
 - 2. Directory Structure
 - 3. Backend Architecture (3.1–3.6)
 - 4. Frontend Architecture (4.1–4.6)
 - 5. Data Flow
 - 6. API Documentation
 - 7. Role-Based Access Control
 - 8. Security Architecture
 - 9. Error Handling
 - 10. Integrations
 - 11. Reporting & Analytics
 - 12. Deployment & DevOps
 - 13. Key Libraries
- Glossary
- Appendix

Executive Summary

Product Overview

The POS System is a full-stack restaurant management platform comprising two independently deployable services: a React 18 SPA (frontend) and a Node.js/Express REST API (backend). The backend uses Prisma ORM against PostgreSQL with JWT-based authentication. The system is containerised via Docker, deployed on Railway, and uses GitHub Actions for CI/CD. Real-time updates are delivered through Socket.IO.

Key Technical Capabilities

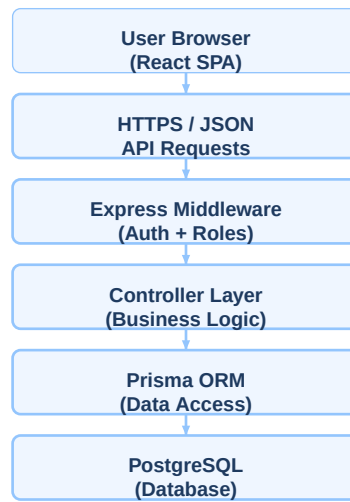
Capability	Implementation	Benefit
Stateless Auth	JWT with 7-day expiry	Horizontal scalability, no session store
Type-Safe DB Access	Prisma ORM	Compile-time query validation, auto-generated types
Real-Time Updates	Socket.IO WebSockets	Instant order status push to all clients
Role-Based Security	Multi-layer middleware	Frontend UI + Backend API enforcement
Modular Architecture	Layered Express design	Maintainable, testable, extensible codebase
Containerised Deploy	Docker + Railway	Consistent environments, automated CI/CD

System Architecture Overview

The system follows a three-tier architecture: Presentation (React SPA), Application (Express API with middleware, controllers, and Prisma), and Data (PostgreSQL). Communication uses HTTPS/JSON with WebSocket fallback for real-time features. External integrations include payment gateways, Twilio SMS, and Nodemailer for email.

Architecture Flow

CONFIDENTIAL



1. Project Overview

Architecture Overview

The POS System is a full-stack restaurant management platform comprising two independently deployable services: a React 18 single-page application (frontend) and a Node.js/Express REST API (backend). The frontend handles all user interface rendering and client-side routing, while the backend manages business logic, data persistence, authentication, and authorization. Communication occurs over HTTPS with JSON payloads, supplemented by Socket.IO for real-time bidirectional updates.

The backend uses Prisma ORM for type-safe database access against a PostgreSQL database. Authentication is JWT-based using the jsonwebtoken library, with passwords hashed via bcryptjs. The system is containerised using Docker and deployed on Railway, with a GitHub Actions CI/CD pipeline that automatically builds and deploys both services on every push to the pos-system branch.

Component	Technology	Port	URL
Frontend SPA	React 18 + Vite 5	8080	https://fmb-mvp.up.railway.app
Backend API	Node.js + Express	8080	https://mvp-fmb.up.railway.app/api
Database	PostgreSQL (Railway)	5432	Managed by Railway

2. Directory Structure

Repository Layout

The repository follows a monorepo pattern with backend/ and frontend/ directories plus root-level configuration.

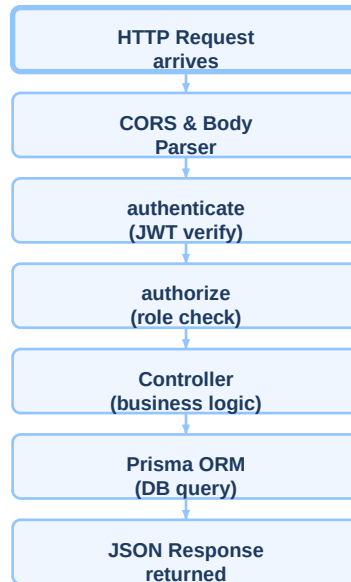
- `docker-compose.yml`
Local development Docker orchestration
- `railway.json`
Railway deployment service definitions
- `.github/workflows/deploy.yml`
CI/CD pipeline configuration
- `backend/src/server.js`
Express entry point
- `backend/src/routes/`
12 route files
- `backend/src/controllers/`
Business logic handlers
- `backend/src/middleware/`
auth.js, roles.js, errorHandler.js
- `backend/prisma/schema.prisma`
Database schema (35+ models)
- `frontend/src/main.jsx`
React DOM entry point
- `frontend/src/App.jsx`
Root component with routes
- `frontend/src/pages/`
13 page-level components

3. Backend Architecture

Layered Architecture

The backend follows a layered architecture: routes define the API surface, middleware handles cross-cutting concerns (auth, roles, errors), controllers contain business logic, and Prisma manages database access. This structure ensures maintainability, testability, and clear separation of concerns.

Request Processing Pipeline



3.1 Entry Point — server.js

Bootstraps Express: loads env vars via dotenv, configures CORS + JSON body parser (15MB limit for base64 images) + cookie parser, mounts routes under /api, initializes Socket.IO, starts HTTP server on PORT (5000 local, 8080 Railway), registers global error handler.

3.2 Routes Layer

- backend/src/routes/auth.routes.js
Authentication, user CRUD (role-gated)
- backend/src/routes/module.routes.js
Module listing and override management
- backend/src/routes/inventory.routes.js
12-section inventory CRUD
- backend/src/routes/order.routes.js
Order CRUD, status updates, code lookup
- backend/src/routes/menu.routes.js
Menu CRUD with category filters
- backend/src/routes/restaurant.routes.js
Restaurant CRUD by building
- backend/src/routes/building.routes.js
Building CRUD
- backend/src/routes/payment.routes.js
Payment processing
- backend/src/routes/delivery.routes.js
Delivery confirmation
- backend/src/routes/dashboard.routes.js
KPIs, charts, reports
- backend/src/routes/food-card.routes.js
Food card management

3.3 Controllers Layer

Business logic in backend/src/controllers/. Each controller validates inputs, enforces rules, interacts with Prisma, emits Socket.IO events where applicable, and returns JSON. Key controllers: auth (login, register, user CRUD with hierarchy enforcement), module (three-tier override resolution), inventory (auto-deduction hook on order completion), order (status management), menu (food category, base64 image handling).

3.4 Middleware

authenticate: Extracts JWT from Authorization header, verifies via jsonwebtoken, attaches { id, role, isSuperadmin, buildingId, restaurantId } to req.user. Returns 401 if invalid. authorize: Checks req.user.role against allowed roles. SuperAdmin always passes. errorHandler: Catches unhandled errors, returns consistent 500 JSON response.

3.5 Database Schema (Prisma)

35+ models in backend/prisma/schema.prisma. Key model groups:

Model	Purpose	Primary Key	Related Tables
User	System user accounts	id (PK)	Building, Restaurant
Building	Physical locations	id (PK)	Restaurant, User
Restaurant	Operational outlets	id (PK)	Building, Menu, Order, User
Order	Customer orders	id (PK)	Restaurant, OrderItem
OrderItem	Line items per order	id (PK)	Order, MenuItem
InventoryItem	Stock items	id (PK)	InventoryCategory, InventoryStock
InventoryStock	Current stock levels	id (PK)	InventoryItem
RecipeMapping	Menu-to-ingredient links	id (PK)	MenuItem, RecipeIngredient
Module	System modules	id (PK)	UserModule, RestaurantModule, BuildingModule

3.6 Authentication & JWT Flow

JWT-based stateless authentication. Payload: { id, role, isSuperadmin, buildingId, restaurantId }. Token expires in 7 days. Every request includes Authorization: Bearer <token>. authenticate() middleware decodes and verifies on each request.

```
function generateToken(user) {  
  return jwt.sign({ id: user.id, role: user.role,  
    isSuperadmin: user.isSuperadmin,  
    buildingId: user.buildingId, restaurantId: user.restaurantId  
  }, JWT_SECRET, { expiresIn: "7d" });  
}
```


4. Frontend Architecture

Component Hierarchy

React 18 SPA built with Vite 5 and Tailwind CSS 3. Component structure: AuthProvider > SocketProvider > React Router Routes. Lazy-loaded pages for code splitting. Dual filtering of sidebar links by role (SIDEBAR_LINKS[role]) and enabled modules (allowedModules).

```
<AuthProvider>
  <SocketProvider>
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route element={<ProtectedRoute />}>
        <Route element={<Layout />}>
          ... lazy-loaded page routes
        </Route>
      </Route>
    </Routes>
  </SocketProvider>
</AuthProvider>
```

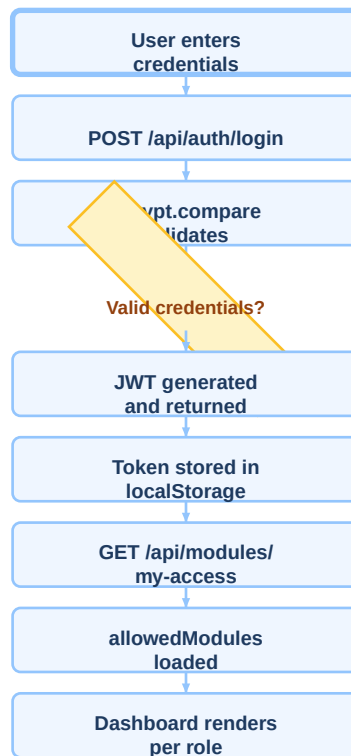
Key Components

Pages (13): Login, Dashboard, Orders, Menu, Inventory, Users, Buildings, Restaurants, ModuleManagement, DeliveryConfirmation, Checkout, OrderSuccess, Register. UI Primitives: Button, Modal, Input, Table, Badge, Card, PageHeader. Layout: Layout, TopBar, Sidebar, BottomNav, ProtectedRoute. Context: AuthContext (auth state, login/logout/switchRole), SocketContext (Socket.IO client). API Layer: Axios client with JWT interceptor + 401 handler, grouped endpoint methods.

5. Data Flow

5.1 Login Sequence

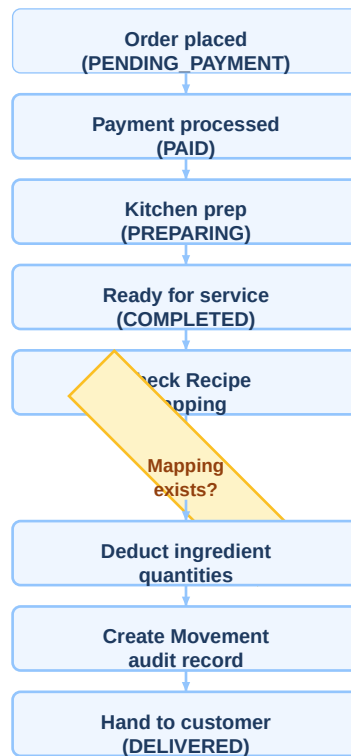
Login Flow



5.2 Order Lifecycle

Orders flow: PENDING_PAYMENT -> PAID -> PREPARING -> COMPLETED -> DELIVERED. On COMPLETED, the system queries RecipeMapping and deducts ingredient quantities from InventoryStock, creating an InventoryMovement audit record. Socket.IO broadcasts all status changes.

Order Lifecycle



5.3 Module Access Resolution

Three-tier override: User-level > Restaurant-level > Building-level > Default (all enabled). Super Admin bypasses all.

```
const result = allModules.filter(mod => {  
  if (mod.id in userOverrideMap) return userOverrideMap[mod.id];  
  if (mod.id in restaurantOverrideMap) return restaurantOverrideMap[mod.id];  
  if (mod.id in buildingOverrideMap) return buildingOverrideMap[mod.id];  
  return true;  
});
```


6. API Documentation

Authentication Endpoints

Endpoint	Method	Auth	Description
/api/auth/login	POST	No	Authenticate user, returns JWT token
/api/auth/register	POST	No	Register new customer account
/api/auth/profile	GET	Yes	Get authenticated user profile
/api/auth/switch-role	POST	Yes	Switch active role (Super Admin only)
/api/auth/users	GET/POST	Yes	List or create users (role-gated)

Login — Sample Request/Response

POST /api/auth/login

Request: { "username": "restmgr1", "password": "restmgr123" }

Response: { "token": "<JWT>", "user": { "id": 2, "username": "restmgr1",
"role": "RESTAURANT_MANAGER", "isSuperadmin": false } }

Error: 401 { "message": "Invalid credentials" }

Order Endpoints

Endpoint	Method	Auth	Description
/api/orders	GET	Yes	List orders (filtered by scope)
/api/orders	POST	Yes	Create new order
/api/orders/:id	GET	Yes	Get order details
/api/orders/:id/status	PATCH	Yes	Update order status
/api/orders/code/:code	GET	Yes	Lookup order by order code

Module & Inventory Endpoints

Endpoint	Method	Description
/api/modules/my-access	GET	Get user's accessible modules
/api/modules/overrides	GET/POST/DELETE	Manage module overrides
/api/inventory/items	GET/POST/PATCH/DELETE	Manage inventory items
/api/inventory/purchase-orders	GET/POST/PATCH	Purchase order management
/api/inventory/goods-receipts	GET/POST	Goods receipt processing
/api/inventory/transfers	GET/POST/PATCH	Stock transfer management
/api/inventory/recipe-mappings	GET/POST/PATCH/DELETE	Recipe mapping CRUD

7. Role-Based Access Control

Access Control Matrix

SA = Super Admin, BM = Building Manager, RM = Restaurant Manager, CF = Chef, CU = Customer.

Feature	SA	BM	RM	CF	CU
Dashboard	Y	Y	Y	Y	Y
Order Management	Y*	Y*	Y	Y	Y**
Menu Management	Y	R	Y	R	B
Inventory Management	Y	Y	Y	L	N
User Management	Y	Y	Y	N	N
Building Management	Y	Y	R	N	N
Restaurant Management	Y	Y	Y	R	R
Delivery Confirmation	Y*	Y*	Y	Y	N
Module Management	Y	N	N	N	N
QR Code Scanner	N	N	Y	Y	N

Y=Full, Y*=Read-only, R=Read, B=Browse, L=Limited, N=No Access

Enforcement Rules

- Users can only manage accounts with roles strictly below their own level.
- BM scoped to assigned building; RM scoped to assigned restaurant.
- Super Admin bypasses all role-level and scope-level restrictions.
- Self-deletion is prevented at the API level.
- Enforcement at both frontend (UI visibility) and backend (API middleware) layers.

8. Security Architecture

Authentication Mechanism

The system uses JWT-based stateless authentication. Upon successful login with valid credentials, the backend generates a signed JWT containing the user's identity and role information. The token is stored client-side in `localStorage` and included in the `Authorization` header of every API request. The backend verifies the token on each request using the `authenticate` middleware, which decodes the JWT and populates the `req.user` object.

Authorization Model

Role-Based Access Control (RBAC) is enforced at every API endpoint through the `authorize` middleware. Each route specifies which roles are permitted, and Super Admin is always allowed. Data scoping is additionally enforced: Building Managers see only their building's data, and Restaurant Managers see only their restaurant's data.

Password Policies

- Passwords are hashed using `bcryptjs` with salt rounds before storage.
- Plain-text passwords are never stored or logged.
- Minimum password length is enforced at account creation.

Session & Token Management

- JWT tokens expire after 7 days from issuance.
- Tokens are self-contained; no server-side session store is required.
- No refresh token mechanism; re-authentication is required after expiry.
- The response interceptor in the Axios client automatically handles 401 responses by clearing `localStorage` and redirecting to the login page.

Audit Trail

- All stock changes create `InventoryMovement` records for complete traceability.
- Order status changes are logged with timestamps and user context.
- User management actions (create, edit, delete) are performed by authenticated users with role hierarchy enforcement.

Data Encryption

- Data in transit: HTTPS/TLS for all HTTP communication between frontend and backend.
 - Data at rest: PostgreSQL database encryption is managed by the hosting provider (Railway).
 - Passwords: `bcrypt` hashing with salt for storage security.
-

9. Error Handling

Error Response Format

All API errors follow a consistent JSON structure: { "message": "Human-readable description", "error": "Technical details (development only)" }.

Error Type	HTTP Status	Description	Example Cause
Validation Error	400	Invalid request payload	Missing required field, invalid format
Authentication Error	401	Missing or invalid JWT	Token expired, invalid signature
Authorization Error	403	Insufficient role permissions	User role not permitted for endpoint
Not Found	404	Resource does not exist	Invalid order code, unknown user ID
Conflict	409	Resource state conflict	Duplicate username, invalid status transition
Server Error	500	Unhandled exception	Database failure, null pointer

Error Handling Strategy

The `global errorHandler` middleware catches all unhandled exceptions. In development, the full stack trace is returned. In production, only a generic message is returned to avoid exposing internal details. Prisma errors are mapped to appropriate HTTP status codes. The frontend `response interceptor` handles 401 by clearing auth state and redirecting to `/login`. All other errors are surfaced via `react-hot-toast` notifications.

10. Integrations

Integration	Technology	Purpose	Configuration
Payment Gateway	Custom endpoint	Process order payments	POST /api/payment/pay controller
SMS Service	Twilio ^4	Send OTP via SMS	TWILIO_* env variables
Email Service	Nodemailer ^6	Send OTP and notifications	SMTP configuration in env
QR Generation	qrcode.react ^4	Generate order QR codes	Frontend library, no config
QR Scanning	html5-qrcode ^2.3	Camera-based QR reading	Browser camera permission
Real-Time	Socket.IO ^4	Push order status updates	Same backend port
Auth	jsonwebtoken ^9 + bcryptjs ^2.4	JWT auth + password hashing	JWT_SECRET env variable

11. Reporting & Analytics

Dashboard Reports

Report	Data Source	Available Filters	User Roles
Order KPIs	Order table	Date range, restaurant	All roles
Revenue Summary	Order + OrderItem	Date range, building	RM, BM, SA
Order Status Distribution	Order status enum	Restaurant, date	RM, BM, SA
Low Stock Alerts	InventoryStock	Restaurant, category	RM, BM, SA
Super Admin Overview	All tables	Building, date range	SA only

Export Options

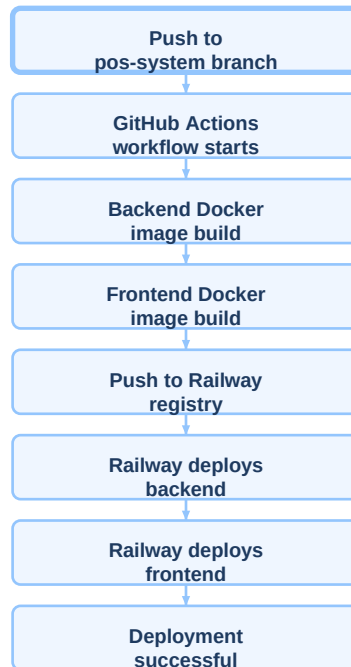
Dashboard data is viewable within the application interface. Future enhancements will include CSV/Excel export capabilities and scheduled report delivery.

12. Deployment & DevOps

CI/CD Pipeline

Push to pos-system branch triggers GitHub Actions workflow: build Docker images for backend and frontend, push to Railway registry, trigger rolling deployment.

Deployment Pipeline



Environment Variables

Backend

```
JWT_SECRET=pos-system-jwt-secret-2024  
CLIENT_URL=https://fmb-mvp.up.railway.app
```

Frontend

```
VITE_API_URL=https://mvp-fmb.up.railway.app/api  
VITE_SOCKET_URL=https://mvp-fmb.up.railway.app
```

Rollback Procedure

- Navigate to Railway dashboard for the affected service.
 - Open the Deployments tab and identify the last known working deployment.
 - Click "Redeploy" on that deployment to restore the previous version.
-

13. Key Libraries & Versions

Backend

Library	Version	Purpose
express	^4.18	HTTP server framework
@prisma/client	^5	Type-safe PostgreSQL ORM
jsonwebtoken	^9	JWT generation and verification
bcryptjs	^2.4	Password hashing
socket.io	^4	Real-time WebSocket server
cors	^2.8	Cross-Origin Resource Sharing
dotenv	^16	Environment variable loading
nodemailer	^6	Email sending (OTP)
twilio	^4	SMS sending (OTP)

Frontend

Library	Version	Purpose
react / react-dom	^18	UI library
react-router-dom	^6	Client-side routing
vite	^5	Build tool and dev server
tailwindcss	^3	Utility-first CSS framework
axios	^1	HTTP client with interceptors
socket.io-client	^4	WebSocket client
html5-qrcode	^2.3	QR code scanner
qrcode.react	^4	QR code generation

Dev Dependencies

Library	Version	Purpose
prisma	^5	Schema management and migrations
nodemon	^3	Auto-restart during development
@vitejs/plugin-react	^4	Vite React plugin

Glossary

Term	Definition
API	Application Programming Interface — a set of protocols for building and integrating applications
CRUD	Create, Read, Update, Delete — four basic operations of persistent storage
ERD	Entity-Relationship Diagram — visual representation of database tables and relationships
GRN	Goods Receipt Note — document acknowledging receipt of goods from a vendor
JWT	JSON Web Token — compact, URL-safe token format for authentication
KDS	Kitchen Display System — digital display showing orders to kitchen staff
KPI	Key Performance Indicator — measurable value showing how effectively objectives are achieved
ORM	Object-Relational Mapping — technique for converting data between type systems
PO	Purchase Order — commercial document issued by a buyer to a seller
POS	Point of Sale — place where a retail transaction is completed
RBAC	Role-Based Access Control — restricting system access to authorised users
SIT	System Integration Testing — testing phase verifying system component interactions
SKU	Stock Keeping Unit — unique identifier for each product or item in inventory
SPA	Single-Page Application — web app that loads one HTML page and dynamically updates
UAT	User Acceptance Testing — final testing phase before deployment

Appendix

Version History

Version	Date	Author	Changes
1.0	June 2026	Engineering Team	Initial release

Known Limitations

- The dashboard does not auto-refresh; manual reload is required for updated KPIs.
- QR scanning requires an active camera and may not function on all mobile browsers.
- The system currently supports Indian Rupees (INR) as the sole currency.

Future Enhancements

- Real-time dashboard auto-refresh via WebSocket push updates.
- Multi-currency support for international deployments.
- Advanced analytics with predictive inventory and demand forecasting.
- Offline mode for order processing during network interruptions.
- Mobile native applications for iOS and Android platforms.

Support Contact

For technical support, report issues at the project repository: <https://github.com/PugazhendhiS-alt/FNB>. For deployment or infrastructure issues, contact the platform operations team through the organisation's IT support portal.